Enterprise RAG Document Assistant - Large Test Manual

This document is intentionally long and structured for testing document upload, text extraction, chunking, embeddings, retrieval, and grounded question answering.

1. Product Overview

The Enterprise RAG Document Assistant is an internal knowledge assistant designed to help employees find reliable answers from authorized company documents. Users can upload PDF, DOCX, and TXT files. The backend extracts readable text, stores document metadata, and prepares the extracted content for retrieval. The system is intended for enterprise scenarios where answers should be based on internal knowledge rather than unsupported model memory. A typical user may upload product manuals, policy documents, support procedures, engineering specifications, onboarding guides, or operational handbooks. The application combines authentication, secure document ownership, conversational AI, and retrieval augmented generation in one architecture.

The Enterprise RAG Document Assistant is an internal knowledge assistant designed to help employees find reliable answers from authorized company documents. Users can upload PDF, DOCX, and TXT files. The backend extracts readable text, stores document metadata, and prepares the extracted content for retrieval. The system is intended for enterprise scenarios where answers should be based on internal knowledge rather than unsupported model memory. A typical user may upload product manuals, policy documents, support procedures, engineering specifications, onboarding guides, or operational handbooks. The application combines authentication, secure document ownership, conversational AI, and retrieval augmented generation in one architecture. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

2. Authentication and Security

Every protected API request requires a valid JWT access token. The token identifies the current user and is sent in the Authorization header using the Bearer scheme. Passwords are never stored as plain text. Passwords are hashed using Argon2 before they are persisted. When a user requests a document, conversation, or message history, the backend verifies ownership before returning the resource. This ownership check is important because enterprise systems must prevent one user from reading another user's private documents or conversations. Authentication answers the question of who the user is, while authorization determines what that user is permitted to access.

Every protected API request requires a valid JWT access token. The token identifies the current user and is sent in the Authorization header using the Bearer scheme. Passwords are never stored as plain text. Passwords are hashed using Argon2 before they are persisted. When a user requests a document, conversation, or message history, the backend verifies ownership before returning the resource. This ownership check is important because enterprise systems must prevent one user from reading another user's private documents or conversations. Authentication answers the question of who the user is, while authorization determines what that user is permitted to access. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

3. Document Upload Process

The document upload endpoint accepts multipart form data. Supported file types are PDF, DOCX, and TXT. The service validates the MIME type and checks that the uploaded file does not exceed the configured maximum size. A UUID-based filename is generated for storage so that two users uploading files with

identical original names do not overwrite one another. The original filename is still stored in the database for display purposes. After validation, the file is written to the local uploads directory and text extraction begins. If extraction succeeds, the document processing status becomes READY. If extraction fails, the status becomes FAILED.

The document upload endpoint accepts multipart form data. Supported file types are PDF, DOCX, and TXT. The service validates the MIME type and checks that the uploaded file does not exceed the configured maximum size. A UUID-based filename is generated for storage so that two users uploading files with identical original names do not overwrite one another. The original filename is still stored in the database for display purposes. After validation, the file is written to the local uploads directory and text extraction begins. If extraction succeeds, the document processing status becomes READY. If extraction fails, the status becomes FAILED. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

4. Text Extraction

PDF extraction is performed page by page. Each page is read and its text is collected into a single extracted text value. DOCX extraction reads document paragraphs, while TXT extraction reads plain UTF-8 text. The extracted text is stored in PostgreSQL so later processing does not need to repeatedly reopen the original file. For PDF documents, the system can also record the number of pages. Scanned image-only PDFs are outside the current project scope because they normally require OCR. Keeping extraction logic separate from the API router makes the code easier to maintain and test.

PDF extraction is performed page by page. Each page is read and its text is collected into a single extracted text value. DOCX extraction reads document paragraphs, while TXT extraction reads plain UTF-8 text. The extracted text is stored in PostgreSQL so later processing does not need to repeatedly reopen the original file. For PDF documents, the system can also record the number of pages. Scanned image-only PDFs are outside the current project scope because they normally require OCR. Keeping extraction logic separate from the API router makes the code easier to maintain and test. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

5. Chunking Strategy

Large documents should not be sent to an embedding model or language model as one enormous block of text. Instead, the extracted text is divided into smaller chunks. In this project, the initial chunk size is approximately 1000 characters with an overlap of 200 characters. The overlap helps preserve context when an important sentence starts near the end of one chunk and continues into the next chunk. Each chunk receives a chunk index and is linked back to its parent document using the document ID. Chunking is a preprocessing step that prepares the content for vector embeddings and semantic retrieval.

Large documents should not be sent to an embedding model or language model as one enormous block of text. Instead, the extracted text is divided into smaller chunks. In this project, the initial chunk size is approximately 1000 characters with an overlap of 200 characters. The overlap helps preserve context when an important sentence starts near the end of one chunk and continues into the next chunk. Each chunk receives a chunk index and is linked back to its parent document using the document ID. Chunking is a preprocessing step that prepares the content for vector embeddings and semantic retrieval. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

6. Why Chunk Overlap Matters

Consider a maintenance instruction that begins at character position 950 and continues beyond character position 1050. Without overlap, one chunk may contain only the beginning of the instruction while the next chunk contains only the ending. With overlap, part of the previous context appears in the next chunk. This increases the chance that semantic search retrieves a complete and meaningful passage. Overlap must still be controlled because excessive overlap creates duplicate information, increases storage usage, and increases embedding cost.

Consider a maintenance instruction that begins at character position 950 and continues beyond character position 1050. Without overlap, one chunk may contain only the beginning of the instruction while the next chunk contains only the ending. With overlap, part of the previous context appears in the next chunk. This increases the chance that semantic search retrieves a complete and meaningful passage. Overlap must still be controlled because excessive overlap creates duplicate information, increases storage usage, and increases embedding cost. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

7. Maintenance Policy

For this sample manual, preventive maintenance should be performed every 30,000 kilometers or every 12 months, whichever occurs first. Maintenance includes inspection of critical components, software diagnostics, fluid checks where applicable, verification of safety systems, and review of logged warnings. If a critical fault is detected, the equipment should not be returned to normal operation until the issue has been assessed by an authorized technician. Maintenance records should include the date, mileage or operating hours, technician name, observed issues, and corrective actions.

For this sample manual, preventive maintenance should be performed every 30,000 kilometers or every 12 months, whichever occurs first. Maintenance includes inspection of critical components, software diagnostics, fluid checks where applicable, verification of safety systems, and review of logged warnings. If a critical fault is detected, the equipment should not be returned to normal operation until the issue has been assessed by an authorized technician. Maintenance records should include the date, mileage or operating hours, technician name, observed issues, and corrective actions. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

8. Warranty Policy

The standard warranty period in this sample manual is 24 months from the original purchase date. Warranty coverage applies to defects in materials and workmanship under normal operating conditions. Damage caused by unauthorized modifications, misuse, accidents, neglected maintenance, or operation outside published specifications may be excluded. Customers requesting warranty service should provide proof of purchase, product identification information, and a description of the reported problem. The warranty statement in this test document is fictional and exists only for RAG testing.

The standard warranty period in this sample manual is 24 months from the original purchase date. Warranty coverage applies to defects in materials and workmanship under normal operating conditions. Damage caused by unauthorized modifications, misuse, accidents, neglected maintenance, or operation outside published specifications may be excluded. Customers requesting warranty service should provide proof of purchase, product identification information, and a description of the reported problem. The warranty statement in this test document is fictional and exists only for RAG testing. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

9. Technical Support Procedure

Before contacting technical support, the user should collect the product identifier, software version, error message, approximate time of failure, and steps that reproduce the problem. Screenshots and relevant log excerpts may also be useful. Support requests should avoid including passwords, private keys, access tokens, or other secrets. The first troubleshooting step is to confirm that the service is reachable and that the user is authenticated. The second step is to review application logs and verify database connectivity. The third step is to isolate whether the issue belongs to the frontend, API layer, database, file storage, extraction pipeline, or model provider.

Before contacting technical support, the user should collect the product identifier, software version, error message, approximate time of failure, and steps that reproduce the problem. Screenshots and relevant log excerpts may also be useful. Support requests should avoid including passwords, private keys, access tokens, or other secrets. The first troubleshooting step is to confirm that the service is reachable and that the user is authenticated. The second step is to review application logs and verify database connectivity. The third step is to isolate whether the issue belongs to the frontend, API layer, database, file storage, extraction pipeline, or model provider. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

10. RAG Retrieval Flow

Retrieval augmented generation begins when the user asks a question. The question is converted into an embedding vector using the same embedding model used for document chunks. The system compares the query vector with stored chunk vectors and retrieves the most similar chunks. These retrieved chunks are assembled into context and passed to the language model together with the user's question. The language model is instructed to answer using the supplied context. If the retrieved context does not contain enough information, the assistant should say that the answer is not available in the provided documents instead of inventing facts.

Retrieval augmented generation begins when the user asks a question. The question is converted into an embedding vector using the same embedding model used for document chunks. The system compares the query vector with stored chunk vectors and retrieves the most similar chunks. These retrieved chunks are assembled into context and passed to the language model together with the user's question. The language model is instructed to answer using the supplied context. If the retrieved context does not contain enough information, the assistant should say that the answer is not available in the provided documents instead of inventing facts. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

11. Vector Search

Vector search works by representing text as numeric vectors. Text with similar meaning should appear closer together in vector space. For example, a query asking when service is required should be semantically close to the maintenance section even if the exact words are different. PostgreSQL with the pgvector extension can store embedding vectors and calculate similarity directly in the database. This allows relational metadata, user ownership, document IDs, and vector search to coexist in the same database.

Vector search works by representing text as numeric vectors. Text with similar meaning should appear closer together in vector space. For example, a query asking when service is required should be semantically close to the maintenance section even if the exact words are different. PostgreSQL with the pgvector extension can store embedding vectors and calculate similarity directly in the database. This allows relational metadata, user ownership, document IDs, and vector search to coexist in the same database. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

12. Citations and Grounding

A trustworthy enterprise assistant should show where an answer came from. Later versions of this project can return document names, chunk identifiers, and PDF page numbers as citations. Grounding means that the generated response is supported by retrieved source content. A grounded system is easier for employees to verify and reduces the risk of hallucinated answers. For security, retrieval must also respect document ownership and authorization before any chunk is given to the language model.

A trustworthy enterprise assistant should show where an answer came from. Later versions of this project can return document names, chunk identifiers, and PDF page numbers as citations. Grounding means that the generated response is supported by retrieved source content. A grounded system is easier for employees to verify and reduces the risk of hallucinated answers. For security, retrieval must also respect document ownership and authorization before any chunk is given to the language model. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

13. Example Questions for Testing

Useful test questions include: What is the warranty period? When should preventive maintenance be performed? Which file types can users upload? Why is chunk overlap used? What information should be collected before contacting technical support? How does vector search work? What should the assistant do when the document does not contain an answer? These questions deliberately target different sections so they can later be used to test semantic retrieval quality.

Useful test questions include: What is the warranty period? When should preventive maintenance be performed? Which file types can users upload? Why is chunk overlap used? What information should be collected before contacting technical support? How does vector search work? What should the assistant do when the document does not contain an answer? These questions deliberately target different sections so they can later be used to test semantic retrieval quality. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.

14. Final Testing Notes

This large sample document should produce multiple document_chunks rows when processed with a chunk size of 1000 characters and an overlap of 200 characters. The exact number of chunks depends on the final extracted character count and whitespace handling. After upload, verify that the document status is READY and query the document_chunks table ordered by chunk_index. Later, after embeddings are added, this same document can be reused to test semantic Top-K search and grounded RAG responses.

This large sample document should produce multiple document_chunks rows when processed with a chunk size of 1000 characters and an overlap of 200 characters. The exact number of chunks depends on the final extracted character count and whitespace handling. After upload, verify that the document status is READY and query the document_chunks table ordered by chunk_index. Later, after embeddings are added, this same document can be reused to test semantic Top-K search and grounded RAG responses. This additional paragraph intentionally repeats and expands the section so the PDF contains enough extracted text to produce many chunks during testing.